WALLET WEB SELF-CUSTODIAL

Especificacion tecnica de seguridad - Version definitiva V9

Estado	Arquitectura consolidada tras revision adversarial
Fecha	22 de septiembre de 2026
Alcance	Wallet 100% web, self-custodial, EVM, Safe 2-de-3
Objetivo	Resistir el compromiso aislado de U, R o S y preservar soberania U+R

Principio central: ninguna de las tres partes U, R o S debe entregar por si sola control completo de los fondos. U+R conserva siempre la salida soberana del usuario.

1. Alcance y modelo de seguridad

La wallet es una aplicacion web self-custodial sobre una Safe con tres owners y threshold 2-de-3. No se utiliza ERC-4337 ni paymaster para el modelo base; S transmite execTransaction y paga gas.

```
U = User Key
R = Recovery Key
S = Server Executor Key

Threshold = 2 de 3

U + S -> operacion asistida normal
R + S -> recovery asistido
U + R -> control soberano
```

U y R pertenecen exclusivamente al usuario.

S pertenece a la infraestructura de la plataforma y es el unico owner con rol especial.

U y R expresan intencion. S valida, cofirma, transmite y paga gas.

S nunca origina por si sola una operacion: toda cooperacion requiere una SafeTx ya autorizada por U o R y una PlatformSession valida.

U + R puede operar sin la plataforma y conserva control soberano de la Safe.

2. Trust domains

```
A. Wallet Frontend  -> controla U
B. Recovery Frontend -> controla R
C. Backend / Signer  -> controla S
```

Los tres dominios son administrativamente independientes.

Comprometer Wallet Frontend no entrega acceso administrativo a Backend/Signer ni Recovery.

Comprometer Backend/Signer no permite desplegar o modificar Wallet Frontend ni Recovery.

Comprometer Recovery no permite modificar Wallet Frontend ni Backend/Signer.

Wallet y Recovery mantienen supply chains separadas para evitar que una unica dependencia o pipeline comprometa U y R.

3. Cuenta centralizada y PlatformSession

La identidad centralizada de la plataforma es una capa distinta de la custodia criptografica. La cuenta puede recuperarse sin recuperar la private key U anterior.

```
Cuenta plataforma -> autenticacion -> PlatformSession
Password U       -> cifrado de U + autenticacion PAKE
Password R       -> cifrado de R
```

Autenticacion de U

```
Password U
  -> Argon2id -> Root_U
       -> HKDF-SHA256("U-encryption-v1") -> K_U
       -> HKDF-SHA256("U-auth-v1")       -> AuthSecret_U

AuthSecret_U -> OPAQUE/PAKE -> PlatformSession
```

El servidor autentica conocimiento del secreto sin recibir Password U. Si el usuario pierde Password U, puede recuperar su cuenta centralizada mediante email, OTP, 2FA o verificacion reforzada; luego crea una nueva U y usa R para reemplazar la anterior.

```
PlatformSession {
  userId,
  safe,
```

```
  platformId,
  sessionPublicKey,
  sessionId,
  issuedAt,
  expiresAt
}
```

Duracion inicial recomendada de sesion sensible: 15 minutos. Las requests criticas se autentican ademas con la session key efimera.

4. User Key U

```
crypto.getRandomValues()
  -> 256 bits
  -> secp256k1 private key U
```

KDF y cifrado

KDF	Argon2id
Salt	128 bits aleatorios
Output	256 bits
Baseline	64 MiB, 3 iteraciones, parallelism 4
Desktop	128-256 MiB cuando sea viable
Cifrado	AES-256-GCM
Nonce	96 bits aleatorios, nuevo por cifrado

AAD de U.enc: formatVersion, platformId, chainId, walletId, role=U, keyAddress y keyVersion.

U descifrada existe solo durante la operacion necesaria y se descarta best-effort al finalizar.

Storage

Default: Google Drive appDataFolder exclusivo de Wallet, con U.enc ya cifrada localmente.

Alternativas: backup cifrado descargable, QR, export manual o custodia propia.

El onboarding exige al menos una forma persistente de recuperar U.

5. Recovery Key R

```
CSPRNG
  -> 256 bits
  -> secp256k1 private key R
```

R se genera en el Recovery Frontend o puede reemplazarse por una wallet externa controlada por el usuario.

```
Password R
  -> Argon2id -> Root_R
       -> HKDF-SHA256("R-encryption-v1") -> K_R

R -> AES-256-GCM -> R.enc
```

Storage default: Google Drive appDataFolder exclusivo de Recovery.

R permanece fuera del flujo cotidiano.

Se recomienda crear y usar R desde un segundo dispositivo mediante QR, aunque no es obligatorio.

Backup humano

Passwords U/R: 5 palabras aleatorias de una wordlist publica, normalizada, versionada e inmutable de 65.536 entradas, equivalente a aproximadamente 80 bits de entropia cuando la seleccion es uniforme.

Backup directo de una private key: 16 palabras de 16 bits mas 1 palabra de version/checksum. Se ofrecen impresion, QR y export cifrado.

6. Registro inicial de R

```
RecoveryEnrollment {
  userId,
  platformId,
  R,
  nonce,
  issuedAt,
  expiresAt
}
```

Recovery firma el enrollment con una Enrollment Key propia. Wallet muestra la direccion R, el usuario la confirma y U firma exactamente su incorporacion. Backend acepta solo el mismo R contenido en el enrollment y en la firma de U. Una R externa exige proof-of-control.

7. Server Executor S

```
Application Backend
      -> Auth / Policy
      -> mTLS + red privada
Signer Shard
      -> Clef o HSM
      -> S
```

Cada Safe utiliza una S distinta.

Los usuarios se distribuyen entre varios signer shards para limitar blast radius.

Cada S mantiene solo ETH operativo para gas.

Clef puede utilizarse inicialmente; la evolucion de maxima seguridad es un HSM propio secp256k1 con S no exportable.

El Backend/Signer no posee credenciales para desplegar o modificar los frontends U/R.

8. Flujo de operacion asistida

```
1. Usuario autenticado
2. Backend valida PlatformSession
3. Frontend construye/reconstruye SafeTx
4. Usuario revisa y confirma
5. U o R firma SafeTx
6. Backend reconstruye SafeTxHash y valida firma + policy
7. Signer reconstruye nuevamente y repite checks criticos
8. S cofirma exactamente la misma SafeTx
9. S transmite execTransaction y paga gas
```

Backend y Signer decodifican la transaccion desde los bytes firmados. No confian en descripciones suministradas por el frontend.

Firmas recibidas fuera de una PlatformSession valida o fuera de un flujo habilitado por la aplicacion se ignoran aunque criptograficamente sean validas.

9. Safe, SecurityGuard y caminos de firma

SecurityGuard tiene codigo inmutable y mantiene on-chain la identidad del executor S. Safe valida threshold y firmas; el Guard clasifica el camino usando exclusivamente los signers que corresponden a las firmas de threshold realmente validadas por Safe, nunca bytes anexados.

Camino asistido

```
msg.sender = S
signers validados = { S, X }
X = owner distinto de S

Resultado: operacion asistida sujeta a Platform Policy + Guard
```

Camino soberano

```
signers validados = { U, R }
S no participa

Resultado: control soberano
```

El camino U+R no esta sujeto a business policies de la plataforma. Mientras el Guard actual siga instalado puede mantener invariantes estructurales; U+R puede cambiar o retirar el Guard mediante una transaccion soberana de la propia Safe.

Invariantes del camino asistido

```
operation       = CALL
safeTxGas       = 0
baseGas         = 0
gasPrice        = 0
gasToken        = address(0)
refundReceiver  = address(0)
```

Los cambios administrativos asistidos de owners, executor, guard, module guard, modules o fallback pasan por los mecanismos autorizados y nunca se habilitan como una operacion ordinaria.

10. Modules, fallback y superficie de ejecucion

```
modules         = none
ModuleGuard     = DenyAllModuleGuard
fallbackHandler = address(0)
```

El camino asistido mantiene modules vacios, DenyAllModuleGuard y fallbackHandler en cero.

DELEGATECALL no esta permitido en el camino asistido.

U+R puede modificar estas configuraciones como expresion de soberania.

11. Policy Engine y PolicyRegistry

V1 - BOOTSTRAP

```
Application Backend policy
+
Signer policy independiente
+
SecurityGuard estructural
```

El Signer utiliza un snapshot firmado de policy y vuelve a verificar target, selector, token, monto, destination, multicall, secuencia y whitelist directamente desde la SafeTx.

Modo endurecido - RESTRICTED

```
BOOTSTRAP -> RESTRICTED   (transicion unidireccional)

RESTRICTED:
default = DENY
solo permisos explicitos
```

Los nuevos permisos se proponen, esperan 24 horas y luego se activan. Guardianes independientes pueden cancelar durante el timelock. PolicyRegistry gobierna el camino asistido, nunca el camino soberano U+R.

12. Autorizacion economica y destinos externos

Toda salida de valor hacia un destino externo elegido por el usuario requiere destination authorization previa. El frontend no puede habilitarla por si solo.

```
Usuario solicita destination D
  -> Backend crea challenge
  -> canal externo muestra D
  -> usuario confirma OTP/challenge
  -> D queda whitelisteada
```

El challenge es single-use y queda ligado a userId, Safe, chainId, destination, action, nonce y expiresAt.

```
Ejecucion posterior =
PlatformSession
+ firma U
+ destination whitelisteada
+ Backend Policy
+ Signer Policy
+ SecurityGuard
```

Autoridades economicas indirectas

Cualquier approve, permit, Permit2, setApprovalForAll, allowance persistente, generic router o mecanismo equivalente que otorgue a un tercero capacidad futura de mover activos se trata como una salida economica. Debe estar explicitamente permitido, acotado y decodificado. Generic/unbounded approvals quedan denegados por defecto.

Los multicalls asistidos se validan subcall por subcall; una subcall desconocida o no decodificable hace fallar toda la operacion.

13. 2FA transaccional

Las operaciones de mayor riesgo pueden exigir un segundo factor adicional a U + PlatformSession + S.

```
2FA challenge ligado a:
Safe
SafeTxHash
accion
token
amount
destination
nonce
```

Factores posibles: email OTP, TOTP, WebAuthn u otro canal independiente. Para alta de nuevos destinos, el canal externo muestra explicitamente la direccion que se habilita.

14. Rotaciones y recovery

```
Cambio asistido de U, R o S:
U o R + S
  -> propose
  -> 48 h timelock
  -> execute
```

Al proponer se notifica por email y dentro de la aplicacion. U actual o R actual pueden cancelar individualmente en cualquier momento durante el timelock.

U perdida o Password U perdido

```
recuperar cuenta centralizada
  -> PlatformSession valida
```

```
  -> generar U'
  -> desbloquear R
  -> R firma reemplazo U -> U'
  -> S valida
  -> 48 h
  -> U' activa
```

R perdida

```
PlatformSession valida
  -> generar R'
  -> U firma reemplazo R -> R'
  -> S valida
  -> 48 h
  -> R' activa
```

Credencial comprometida que veta su propia rotacion

El procedimiento excepcional es manual: verificacion humana reforzada, doble aprobacion interna y registro auditable; la credencial sana firma una transferencia y S la cofirma hacia una nueva Safe. La Safe anterior se abandona una vez migrados los activos.

15. Nonce, vigencia y cancelacion

Una SafeTx firmada permanece valida hasta su ejecucion o hasta que el usuario la cancele consumiendo el mismo nonce. Los deadlines off-chain son herramientas operativas, no una garantia fundamental de seguridad.

```
TX pendiente: nonce = N

Cancelacion:
NoOperation con nonce = N
  -> ejecucion
  -> nonce N consumido
  -> TX anterior invalida
```

16. WalletFactory, SafeBootstrap y deploy verificable

WalletFactory es inmutable y recibe unicamente U, R, S y salt. Toda la configuracion sensible queda fijada en codigo; el backend no aporta calldata arbitraria.

```
WalletFactory
  -> SafeProxyFactory exacta
  -> Safe singleton exacta
  -> SafeBootstrap exacto

Config fija:
threshold = 2
SecurityGuard
DenyAllModuleGuard
RotationTimelock
PolicyRegistry
executor = S
fallbackHandler = 0
paymentToken = 0
payment = 0
paymentReceiver = 0
```

Safe release, singleton, factory y runtime bytecode hashes quedan fijados por version. Los contratos propios criticos tambien se verifican por address y codehash.

Verificacion previa y posterior

```
Antes:
```

```
initializer hash
expected Safe address

Despues, lectura directa on-chain:
Factory/codehash correctos
Safe implementation/codehash correctos
owners U/S/R correctos
threshold = 2
executor = S
SecurityGuard correcto
ModuleGuard correcto
modules = none
fallbackHandler = 0
RotationTimelock correcto
PolicyRegistry correcto
chainId correcto
```

```
CREATING -> DEPLOYING -> VERIFYING -> ACTIVE
```

La direccion solo se presenta como wallet operativa despues de ACTIVE. Deploy e inicializacion son atomicos.

17. Hardening de frontend y supply chain

Separacion Wallet / Recovery

Repositorio	separado
Dependency tree	separado
Lockfile	separado
CI runner	separado
Build pipeline	separado
Hosting/DNS	separado
OAuth	separado
Release keys	separadas
appDataFolder	separado

Recovery mantiene un frontend minimo. Las dependencias criptograficas comunes usan versiones exactas, hashes fijados y revisiones independientes. Los upgrades comunes no se despliegan simultaneamente en ambos dominios.

Integridad de releases

- Builds reproducibles y assets self-hosted.
- CSP estricta, SRI, Trusted Types, lockfiles fijados y SBOM por build.
- Sin third-party JavaScript en origins criptograficos.
- Service Workers deshabilitados en Wallet y Recovery para evitar persistencia de codigo comprometido.
- Manifest firmado con releaseId, commit y SHA-256 de HTML, JS, WASM, CSS y assets.
- Monitores independientes comparan el deploy con el manifest; una discrepancia pausa la cooperacion de S para esa plataforma.

18. Control soberano y desaparicion de la plataforma

```
U + R
  -> control completo de Safe
  -> cualquier relayer con gas puede transmitir
```

El camino U+R no depende de PlatformSession, Backend, Signer, Policy Engine ni PolicyRegistry de negocio. Con copias validas de U y R, el usuario mantiene control aunque la plataforma desaparezca.

19. Supuestos autoritativos para auditorias

U y R tienen igual poder como owners del usuario.

U+R representa soberania completa y puede cambiar la configuracion de la Safe.

S es el unico owner con rol especial de executor.

Wallet Frontend, Recovery Frontend y Backend/Signer son trust domains administrativos independientes.

Toda cooperacion de S exige PlatformSession valida y una SafeTx previamente autorizada por U o R.

S acepta solo flujos tipificados por la aplicacion; firmas arbitrarias inyectadas fuera de esos flujos se ignoran.

Toda salida externa requiere destination authorization previa.

U o R actuales pueden cancelar individualmente una rotacion pendiente.

Una credencial comprometida que bloquee su sustitucion puede resolverse migrando fondos a una nueva Safe.

Las SafeTx permanecen validas hasta ejecucion o consumo de nonce.

El compromiso conjunto de dos owners suficientes para alcanzar threshold otorga el poder correspondiente por diseno.

20. Matriz minima de assertions de seguridad

ID	Assertion
A1	U sola nunca ejecuta una SafeTx asistida.
A2	R sola nunca ejecuta una SafeTx asistida.
A3	S sola nunca alcanza threshold.
A4	U+S solo ejecuta acciones permitidas por policy y Guard.
A5	R+S solo ejecuta recovery permitido o el flujo excepcional autorizado.
A6	U+R puede operar sin Backend/S y no queda sujeto a business policies.
A7	El Guard clasifica el camino solo con signers correspondientes al threshold validado.
A8	DELEGATECALL, modules y fallback quedan cerrados en el camino asistido.
A9	Una salida externa a destination no autorizada falla aunque U este comprometida.
A10	Approvals/permits/operator rights se tratan como autoridad economica y fallan por defecto si no estan tipificados.
A11	Una Safe con configuracion distinta a Factory/Bootstrap no alcanza ACTIVE.
A12	Una firma no puede reusarse con otro nonce, Safe o chain.
A13	La cancelacion con nonce N invalida cualquier SafeTx pendiente con nonce N.
A14	Comprometer un unico trust domain U, R o S no entrega control completo.

Invariante final

```
U sola -> insuficiente
R sola -> insuficiente
S sola -> insuficiente

Wallet comprometido -> insuficiente
Recovery comprometido -> insuficiente
Backend/S comprometido -> insuficiente

U + S -> operacion de plataforma
R + S -> recovery
U + R -> soberania
```